

pharus Quick Setup Guide

Scope

This document guides through an example setup of pharus and covers every necessary step along the way. It shows how to decide about a fitting coordinate system, define tracking space dimensions and position sensors around it. Defining measured points within the tracking space simplifies setup, calibration and potentially necessary recalibrations during operation. Throughout the process, many *pharus* GUI elements are covered as well.

This setup guide assumes laser ranger devices which are supported by *pharus*, are already properly setup and ready to use.

This document is valid for *pharus* versions from v2.2.x and above. It will have limited applicability for future v3.x releases.

Introduction

In this sample setup we assume an area of roughly 5m by 6m in the Ars Electronica Futurelab's studio - covered with dark-grey floor tiles - as our future interactive dance floor.

For this task, two *Sick LMS100* lidars were prepared, with the intention of one sensor to be placed along the left edge of the space, one along the far edge. To measure distances we have a laser measurement device at hand, a (long) measurement tape is perfectly suitable as well.

Locating Devices & Coordinate System

pharus is often integrated in projects where a coordinate system is externally provided. In our case we're free to decide. In general, *pharus* uses a metric coordinate system with the x axis facing to the right and the y axis upwards in the visualization.

To make our lives easy, we decide the origin of our coordinate system to be in the lower-left corner of the tracking area. Furthermore, we plan to set up the two sensors in the center of the left and the top edge.



Figure 1: The example space (dark flooring) - an area of roughly five by six meters.



Figure 2: The ingredients for this setup. Two Sick LMS100 lidars and a measurement device

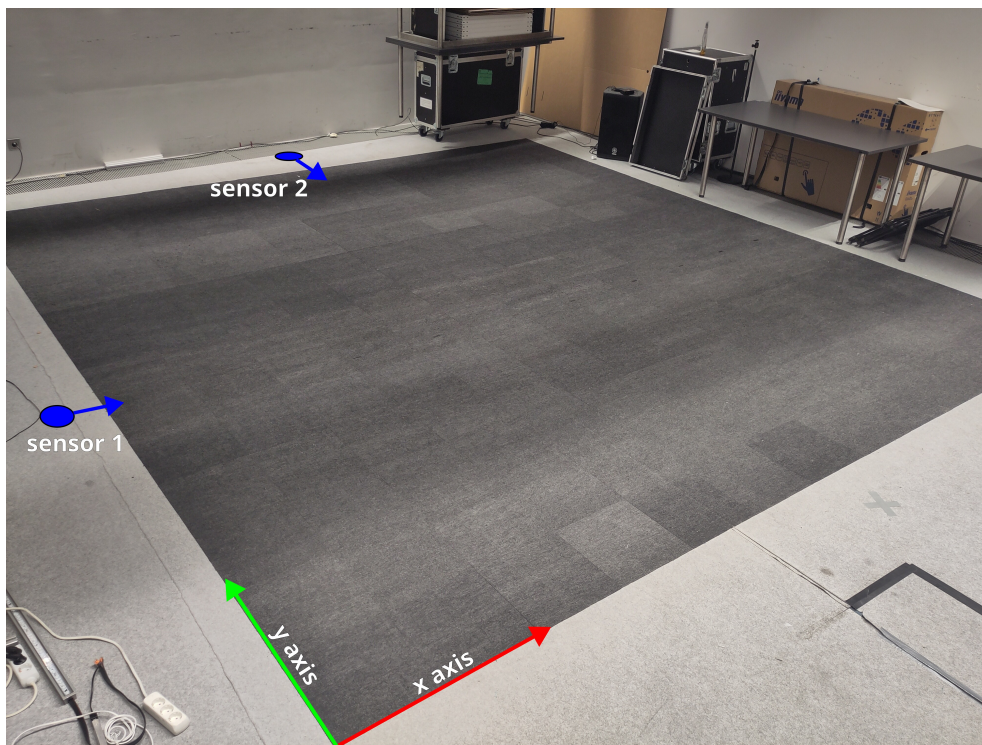


Figure 3: Intended layout. Coordinates origin in the bottom-left corner, x-axis facing to the right, y up. One sensor from the left, one from the top.

Taking Measurements

With an envisioned setup in mind, we start measuring the most obvious: our tracking space, which is called *Observation Space (OSpace)* in *pharus* terminology.

From the origin in the lower-left corner, we take our measurement device (tape or laser) and measure the dimension along the x and y axes.



Figure 4: Measuring dimensions with a laser tool.

Quite often, the floor geometry can simplify our measurements. There might be a grid, some helpful lines or, as in this case: floor tiles. Not only do they facilitate a measuring along the axis, due to their fixed edge length, many measurements can simply be taken by just counting the tiles. In most cases, this is sufficiently precise for an initial setup.

We note down the width and the height of our *OSpace* and continue to *roughly* measure the positions of our two sensors which we mark with eg. a tape on the ground. Note that in our example, sensor 1 already has a negative x axis as it's to the left of our origin.

Simplifying Calibration: Fixes

You might be surprised by the previous instruction stating it's good enough to *roughly* measure the position of a sensor. That is due to the fact that it's often barely possible to precisely determine the position of a sensor. Even so, a precise location is not much worth without exactly knowing the sensor's rotation in space. Due to the nature of angular measurements, with a rotation only being off by some milli rads, the positional error increases quickly with distance.

During calibration, *pharus* thus follows the idea to less care about a precisely measured sensor position and rotation initially. A good system precision will be accomplished iteratively, by aligning the sensor's scan output with measured positions **within OSpace**. That way, the sensor's real position/rotation is determined implicitly.

Such measured positions are called *Fixes* in *pharus*. They usually stay visible (marked) throughout the active setup to be able to quickly recalibrate in case a

sensor was moved or has to be replaced. During calibration, objects will be placed on these marked positions to make them visible in the sensor's scan image.

Choosing Fix Positions

In a room-sized setup usually about three Fixes evenly distributed throughout the envisioned *OSpace* are sufficient. Adjust for larger setups accordingly. It is important to make sure that two Fix positions are never along a line-of-sight from a sensor (in other words: make sure that an object placed on one Fix would occlude an object on another Fix position for a sensor).

In our sample setup, we choose three positions and mark them with tape.



Figure 5: Positions of the defined fixes, marked with red tape.

We work along floor tile corners to determine their position by counting the tiles, else we use the tape and measure their positions from the origin.

pharus - GUI Overview

With the dimensions of our *OSpace* and the coordinates of the three fixes, we launch *pharus*. In the default configuration, we're greeted with a top-down view of a DeepSpace-like setup.

At the top is *pharus*' menu bar to controll the app's various features. The GUI covers all aspects for every-day use - some other more rarely used features are available, although only via the configuration file.

The main view is the visualization of the tracking area, covered by a meter-spaced grid. Most notable are two colored rectangles. The pink one has already been introduced as our Observation Space (*OSpace*), defining the actual tracking space. In other words, only those portions of a sensor's scan that lie within this rectangle are processed further, the others discarded. The second rectangle in a pale yellow is called the Mapping Space (*MSpace*) that can be useful to precisely map a pharus track to a floor projection. For this example setup, we'll set *MSpace* to the same coordinates as *OSpace*.

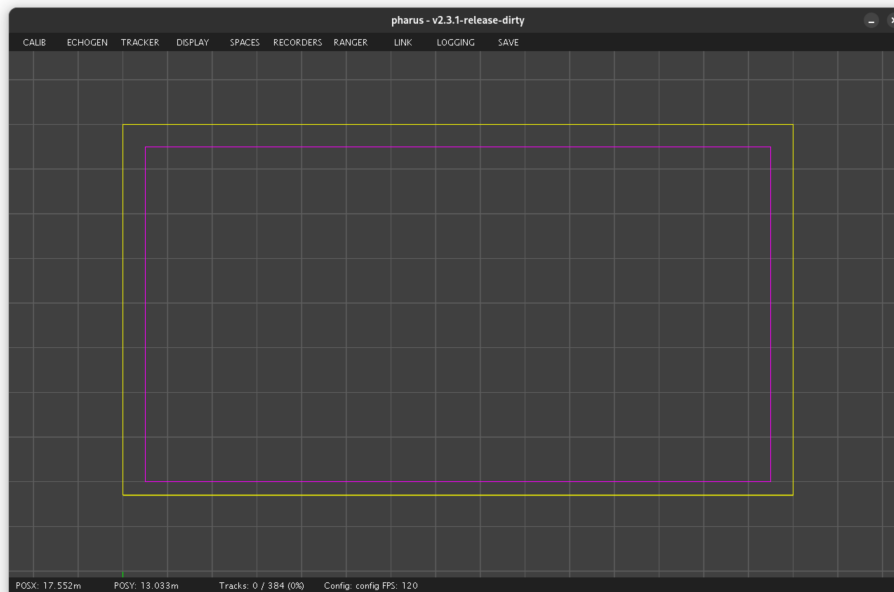


Figure 6: An empty pharus setup.

The bottom line shows the current mouse position converted to world coordinates (meters from the origin), the track load and current frame rate. If applicable, an info about a warning message is displayed. It's then recommended to check the logging window at the menu bar.

Preparations

The next step is to enter the data collected beforehand. Let's start with the Observation Space. Click to SPACES in the menu bar. This opens a set of spin boxes to enter the dimensions of both *OSpace* and *MSpace*. As our tracking space's lower-left corner is also the origin, we can already set the *left* and the *lower* boxes to 0.0 . Into the *right* portion, enter the dimension of our measured value for the x axis. The y axis goes into the *upper* field. Enter the same values for MapSpace.

Once done, both spaces have changed accordingly in the visualization. Press **Shift+R** to re-center the view at the center of *OSpace*.

Close the SPACES window and open DISPLAY next.

This window obviously controls what's visible in the visualization. For many purposes and especially for calibration it can be useful to deselect some features and unclutter the view. The lower half of the window is about Fixes. This is where we enter the coordinates of our measured reference points. Under *Add Fix* type in the x and y coordinated measured. Clicking on **ADD FIX** adds this position to be box above and adds a light-grey hexagon to the visualization with the Fix ID in the center. Repeat for the remaining two positions.

To already prepare the visualization for the calibration process, in the top half we

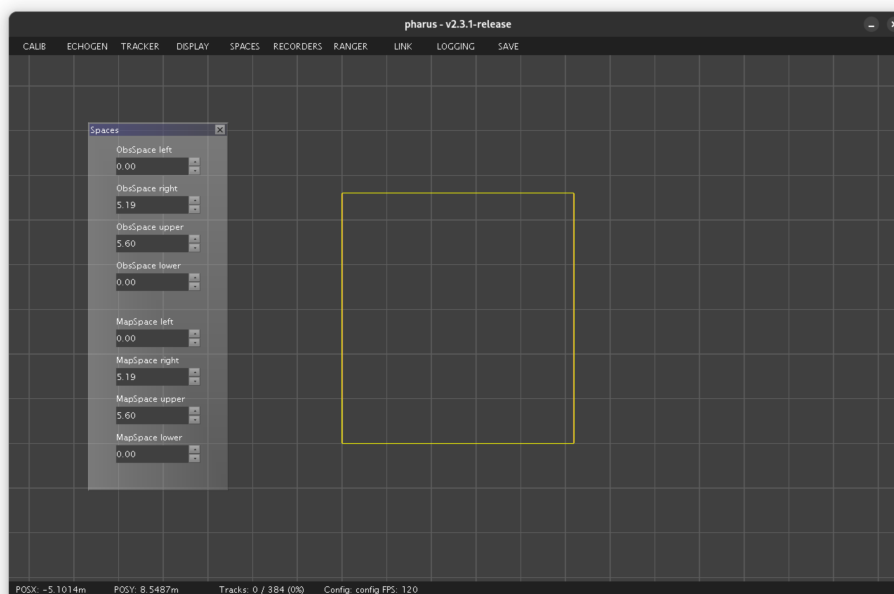


Figure 7: Entering the dimensions of our tracking space.

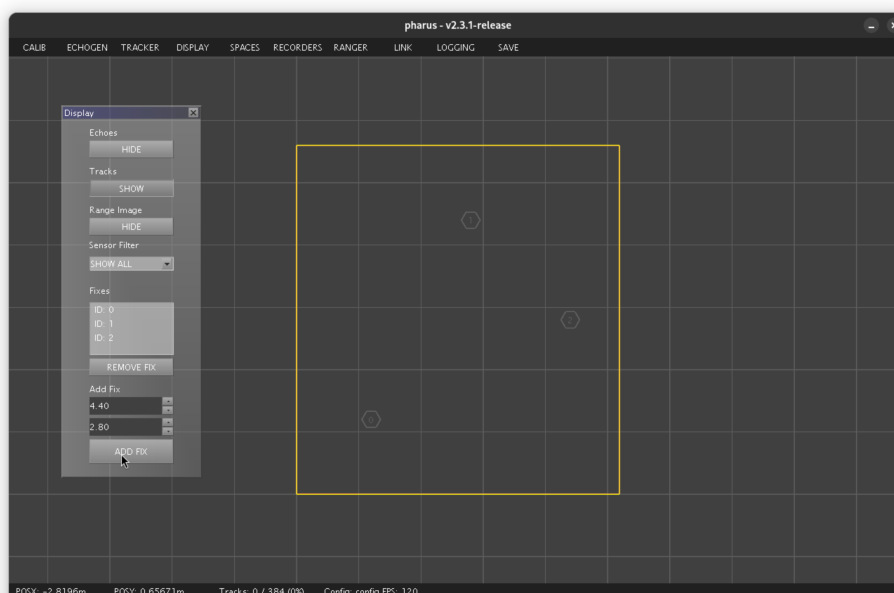


Figure 8: After entering the measured Fix positions, visually confirm that their displayed positions match with actual ones.

deselect features we don't need while confirming that required ones are active. Set *Echoes* and *Tracks* to HIDE and *Range Image* to SHOW.

With this set, we're ready to add the sensors to our configuration and begin with the calibration.

Calibration

In order to perform the calibration process, we need to have something at the Fix positions which can be detected by the sensors. A variety of objects are suitable for the task, it is recommended to use a rather narrow cylindrical object that is high enough to reach into the sensor's scan plane. In our sample setup we use three pieces of aluminum profiles.



Figure 9: Markers (aluminum profiles in this example) placed on top of the Fix positions.

Keep the objects nearby. That way it's easier to quickly recalibrate if necessary.

With the visualization prep and basic tracking area definition from the previous chapter we're all set to start with the calibration. For this process it is assumed that the sensors are configured for use with *pharus* and their IP addresses or serial ports known. For questions around device configurations, contact future-lab@ars.electronica.art.

Roughly measure each sensor's position. Usually it's already enough to guesstimate with the visualization: just move the mouse to the approximate position and note down the pos reading from the bottom bar.

Next, open the RANGER window. First, select the type of ranger to connect to. If the model is a ranger from the manufacturer *Sick*, select SICK BINARY. *Hokuyo* sensors are supported as well, select the corresponding entry. In the field below, enter the device's IP address. If the device is connected via a serial (USB) connection, find out its COM port (Windows) or dev path (Unixoids) in a similar form of `/dev/tty[...]`. This is followed by two fields to enter the sensor's position we guessed earlier. After that, *pharus* needs to know the initial orientation of the device (the direction it looks at). This is set in the *Angle* field. To simplify this, a little helper diagram can be found below. Read it in a way that the sensor is in the center

“looking” in a certain direction. For instance, if your sensor’s to the left of *OSpace* facing to the right, set it to 0.75 .

If for some reason the sensor is mounted upside down, tick the **FLIPPED** box. Finally, if we had multiple Observation Spaces (expert mode), the sensor could be assigned to a specific space with *OSpace ID*.

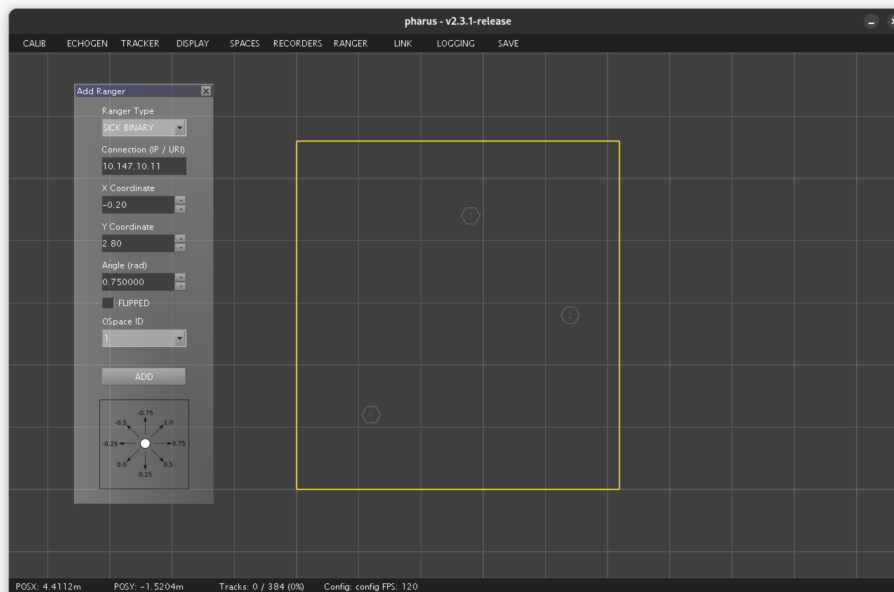


Figure 10: Adding a sensor to the setup.

Click on **ADD** to initiate a connection to the sensor.

Sensor Alignment

Upon successfully connecting to the sensor, we should see a view similar to the one below. If this is not the case, either check view filters set in **DISPLAY** or take a look at the **LOGGING** window in case the connection attempt failed.

We now see the left sensor added to the setup. At the lidar’s position additional info about its placement, connection & state are displayed. Visually we can quickly confirm that our initial alignment is not too far off: we see detections in the proximity of each Fix. The top and right one seem to be detected a tad too low, the left one a bit off to the left. Seeing that some are off to one direction while others to another reveals that it is not (only) a positional but a rotational error as it is nearly impossible to initially guess the device’s orientation. As a general rule of thumb, we correct the rotation (angle) first as far as possible, then a positional offset.

For this, we open the **CALIB** window. There, we select the sensor from the list box, its current placement and angle are updated in the boxes below. Try out the arrows at the *Angle* spin box and see how the range image starts adapting to our input. If our initial position is good already you should be able to bring the range image to

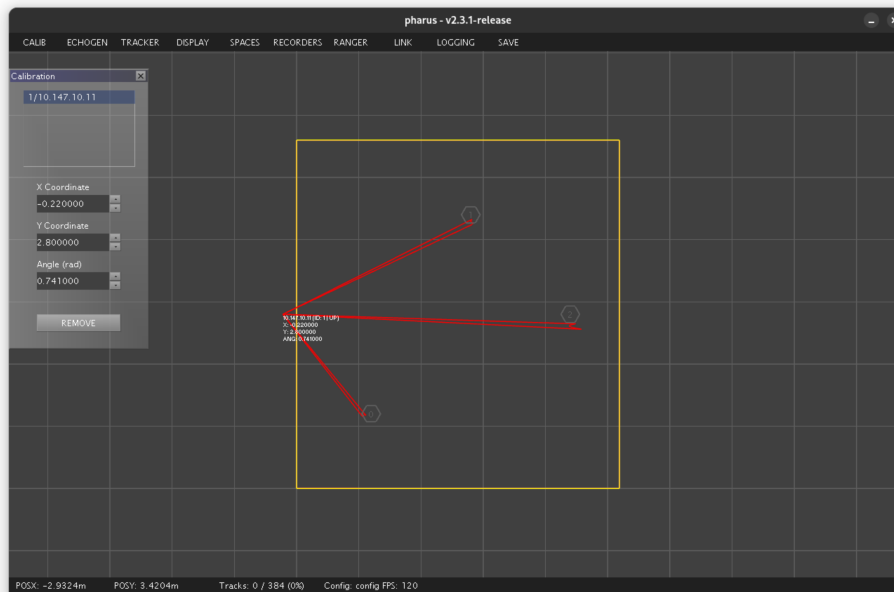


Figure 11: Initial phase of sensor calibration. The first estimate is already close to the target calibration.

an alignment with the Fixes. Else, just rotate the image to a point where all three detections are similarly offset from their Fixes.

In the next step, we adjust the x and y coordinates to bring the scan and the Fixes to an alignment. If we're still not there yet, we repeat rotational & positional alignment until we're satisfied with the result.

In the above screenshot, the alignment process is complete. Next up is the integration of the second sensor.

Adding In Sensors

Little surprising, adding the next sensor to the arrangement follows the same steps. Again, decluttering the visualization supports a good sensor alignment: during the process it is very helpful to disable the visualization of range images from other devices. Under DISPLAY -> *Sensor Filter*, we select the one we're currently calibrating to minimize distraction from others.

Following the process described above, firstly the rotation is corrected, then the position. We now have a good calibration on the second sensor.

With the calibration of our tracking setup complete, we can lift the *Sensor Filter* in DISPLAY and see a good convergence of detection:

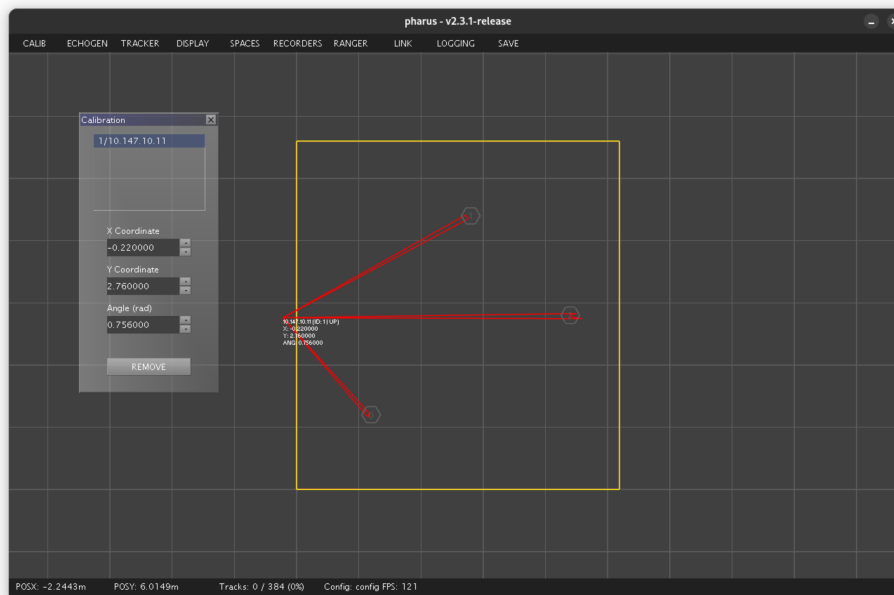


Figure 12: Finalized sensor calibration. The detected markers match the actual position of Fixes.

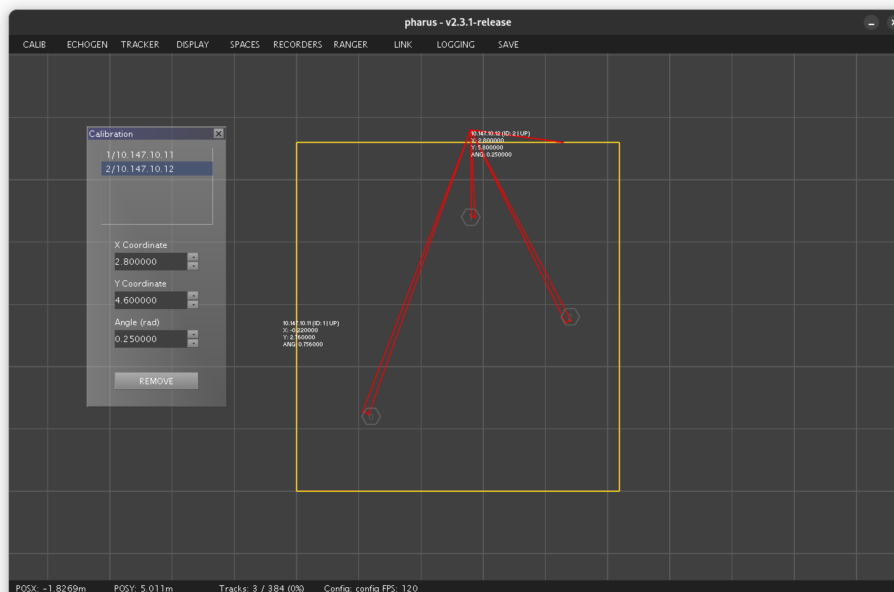


Figure 13: Second round of calibration. Again, the initial alignment is no too far off.

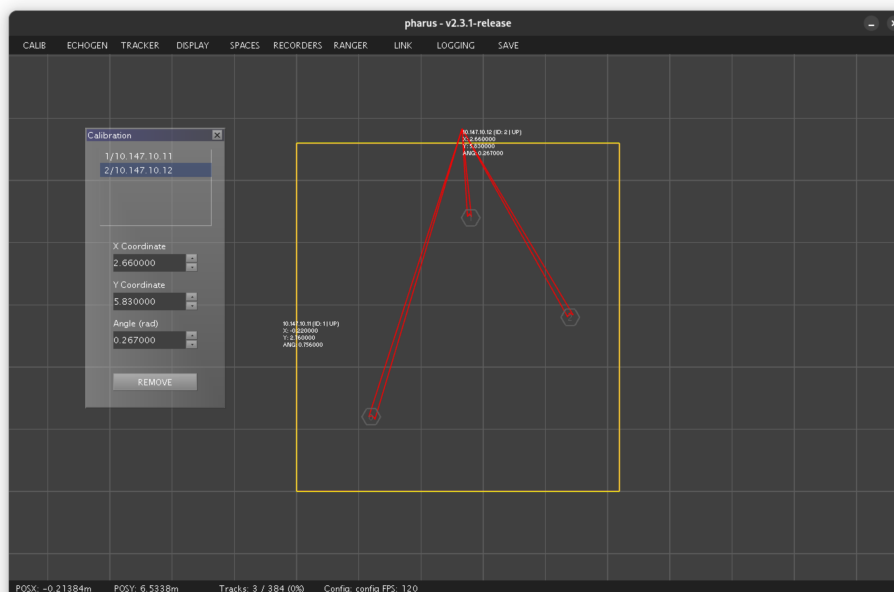


Figure 14: The finalized alignment.

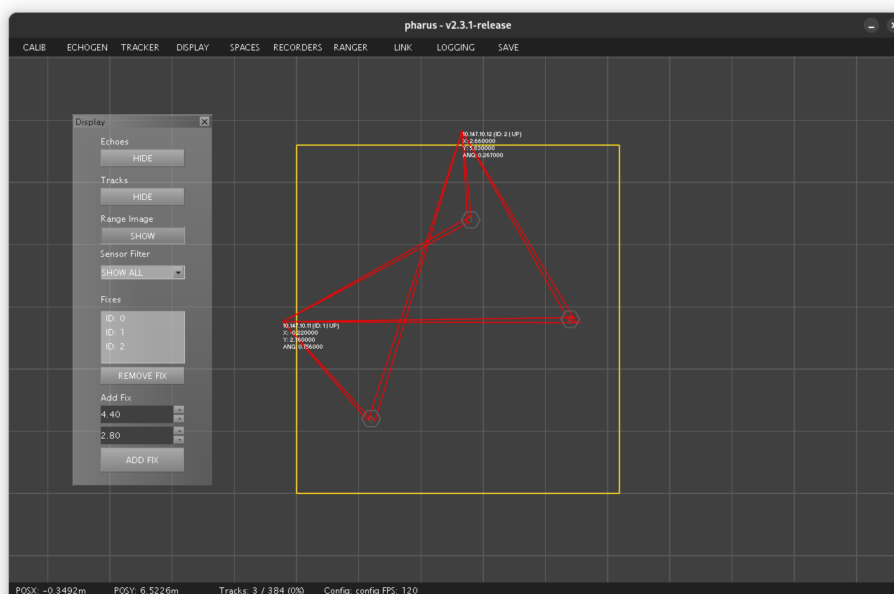


Figure 15: Confirming convergence of both sensors.

Working With The Setup

It's now a good time to save the configuration. In the SAVE window, click the button. *pharus* saves the current setup under `cfg/config.xml`, any existing file will be backed up with a timestamp, in case a config needs to be recovered.

Next, we prepare the visualization for tracking. As stated earlier, in the DISPLAY window, lift existing *Sensor Filters* and re-enable the visualization of *Tracks*. Optionally, enable the display of *Range Images* and *Echoes*.

Now it's time to give it a try and walk around in the setup. You should see yourself as a turquoise circle (the track) with a unique ID in the center. As you walk around you see a trail visualizing your past locations. If you walk fast enough ($>0.25\text{m/s}$), *pharus* will infer a heading from your movement which is shown as a line within the track. If you enabled *Echoes* for the visualization, little grey or white circles follow the position of your feet. While you're at it, remove the detection marker objects from the Fixes.

Connectivity

pharus supports a variety of network protocols to deliver tracking information to other applications. The two most used ones are Tuio and *TrackLink*, a custom, open UDP protocol designed for *pharus*, which is widely used among DeepSpace dev kits and deriving applications. Furthermore, *Websockets* and plain *OSC* are supported as well, although they need to be managed via the config file.

In the LINK window, we can manage multiple Tuio receiver endpoints. Below, *TrackLink* can be enabled or disabled. If `USE_MULTICAST` is checked, multiple endpoints can receive tracking data at the same time, no need to enter their IPs in *pharus*.